Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Мозговая Ж.Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 22.10.26

Москва, 2026

Постановка задачи

Вариант 12

Наложить K раз фильтры эрозии и наращивания на матрицу, состоящую из вещественных чисел. На выходе получается 2 результирующие матрицы.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, должно задаваться ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, с помощью стандартных средств операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Общий метод и алгоритм решения

Программа реализована на языке Си для ОС Linux (запускалась в WSL). Для создания потоков использована библиотека POSIX threads (pthreads).

Использованные функции:

- int pthread_create(...) — создаёт поток, выполняющий заданную функцию.
- int pthread_join(...) — ожидает завершения потока (синхронизация).
- int clock_gettime(...) — точный замер времени выполнения.
- void *malloc(size_t) / void free(void *) — выделение и освобождение памяти.
- Разбор аргументов командной строки для задания размера матрицы (-n), размера ядра (-w), итераций (-k) и лимита потоков (-t).

Алгоритм работы программы:

- Создается матрица вещественных чисел (double) размером $N \times N$.
- Реализуются две операции: **эрозия** (замена элемента на минимальное значение в окне $w \times w$) и **наращивание/дилатация** (замена на максимальное значение).
- Алгоритм распараллелен методом разделения по данным (Data Parallelism): матрица разбивается на горизонтальные полосы. Каждый поток обрабатывает свою полосу строк.
- Так как потоки работают с непересекающимися участками памяти, гонок данных не возникает. Синхронизация происходит только при помощи pthread_join в конце, когда главный поток ожидает завершения всех остальных.
- Ограничение -t контролирует максимальное число потоков.
- После расчетов выполняется проверка корректности: сравниваются результаты параллельного и последовательного алгоритмов (выводится match=yes).

В рамках работы требовалось реализовать многопоточную программу на языке Си, которая накладывает K раз фильтр эрозии и K раз фильтр наращивания (дилатации) на матрицу

вещественных чисел, и исследовать зависимость ускорения и эффективности от входных данных и числа потоков.

Для этого было сделано следующее:

1. **Реализованы две морфологические операции** над матрицей double размером N×N:
    - эрозия — каждый элемент заменяется на минимум в окне w×w;
    - дилатация (наращивание) — каждый элемент заменяется на максимум в окне w×w. Границы обрабатываются «зажимом» индексов (clamp): выход за пределы матрицы заменяется крайним допустимым индексом, что соответствует продлению края (replicate padding).
2. **Распараллеливание выполнено методом разделения по данным (Data Parallelism).** Матрица разбивается на горизонтальные полосы строк, и каждый поток обрабатывает свою полосу. Так как потоки пишут в непересекающиеся участки выходной матрицы и читают только из входной, гонок данных не возникает — синхронизация не требуется, кроме ожидания завершения потоков в конце итерации.
3. **Создание потоков — стандартными средствами ОС**, через POSIX threads (pthread_create, pthread_join). Ограничение максимального числа одновременно работающих потоков задаётся ключом -t; число потоков на каждой итерации не превышает этого значения.
4. **Замер времени** выполняется через clock_gettime(CLOCK_MONOTONIC, ...) с точностью до миллисекунд (дробная часть — микросекунды). Для снижения шума каждая версия (последовательная и параллельная) запускается reps раз (ключ -r), и берётся минимальное время.
5. **Проверка корректности**: после параллельного расчёта результат поэлементно сравнивается с результатом последовательного алгоритма (допуск 1e-9), выводится match=yes/no.
6. **Демонстрация числа потоков** — ключ -d выводит строки вида [thread] id=... rows=[r0,r1) op=..., по которым видно, сколько потоков реально работало и какие полосы они обработали. Дополнительно количество потоков можно наблюдать стандартными средствами ОС (например, ps -L, top -H, htop) во время работы программы.

Алгоритм:

1. Выделяются четыре матрицы: a, b (для последовательного расчёта) и a2, b2 (для параллельного), заполняются одинаковыми псевдослучайными числами (srand(12345) — фиксированное зерно для воспроизводимости).
2. Последовательный алгоритм: K раз эрозия с перестановкой буферов a↔b, затем K раз дилатация. Время суммируется.
3. Параллельный алгоритм: на каждой из 2K итераций вызывается run_parallel_op, которая делит rows строк на nthreads полос (первые rows % nthreads потоков получают на одну строку больше), создаёт потоки pthread_create, каждый выполняет worker со своей структурой Task, затем главный поток делает pthread_join для всех. После каждой итерации буферы меняются местами.

4. Сравнение результатов и вывод метрик: sequential_ms, parallel_ms, speedup = Ts/Tp, efficiency = speedup / T.

Важно: число потоков на каждой итерации фиксировано и равно -t, поэтому ограничение «максимального количества потоков, работающих одновременно» соблюдается автоматически — потоки создаются и завершаются в пределах одной итерации

Для оценки качества распараллеливания используются две метрики.

Время исполнения T_p — время выполнения параллельной версии алгоритма (измеряется clock_gettime(CLOCK_MONOTONIC)), T_s — время последовательной версии. Для снижения шума берётся минимум по reps прогонам.

Ускорение (speedup) — во сколько раз параллельная версия быстрее последовательной:

$$S(T) = \frac{T_s}{T_p}$$

где T_s — время последовательного алгоритма (мс), T_p — время параллельного алгоритма на T потоках (мс). Идеальное значение S(T) = T; на практике S(T) < T из-за накладных расходов на создание/синхронизацию потоков и неидеального баланса.

Эффективность (efficiency) — какая доля теоретического максимума ускорения реально достигнута:

$$E(T) = \frac{S(T)}{T} = \frac{T_s}{T \cdot T_p}$$

где T — число потоков, S(T) — ускорение, T_s, T_p — времена (мс). Значение E лежит в диапазоне (0, 1]: E = 1 означает идеальное линейное ускорение, E → 0 — распараллеливание неэффективно (накладные расходы доминируют).

Код программы

task.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>
#include <math.h>

#define OP_ERODE  0
#define OP_DILATE 1

typedef struct {
    double *in;
    double *out;
    int rows;
    int cols;
    int w;
    int r0;
```

```
    int r1;
    int op;
} Task;

static void erode_region(double *in, double *out, int rows, int cols, int w, int r0,
int r1) {
    int half = w / 2;
    for (int i = r0; i < r1; i++) {
        for (int j = 0; j < cols; j++) {
            double mn = 1e300;
            for (int di = -half; di <= half; di++) {
                int ii = i + di;
                if (ii < 0) ii = 0;
                if (ii >= rows) ii = rows - 1;
                for (int dj = -half; dj <= half; dj++) {
                    int jj = j + dj;
                    if (jj < 0) jj = 0;
                    if (jj >= cols) jj = cols - 1;
                    double v = in[ii * cols + jj];
                    if (v < mn) mn = v;
                }
            }
            out[i * cols + j] = mn;
        }
    }
}

static void dilate_region(double *in, double *out, int rows, int cols, int w, int r0,
int r1) {
    int half = w / 2;
    for (int i = r0; i < r1; i++) {
        for (int j = 0; j < cols; j++) {
            double mx = -1e300;
            for (int di = -half; di <= half; di++) {
                int ii = i + di;
                if (ii < 0) ii = 0;
                if (ii >= rows) ii = rows - 1;
                for (int dj = -half; dj <= half; dj++) {
                    int jj = j + dj;
                    if (jj < 0) jj = 0;
                    if (jj >= cols) jj = cols - 1;
                    double v = in[ii * cols + jj];
                    if (v > mx) mx = v;
                }
            }
            out[i * cols + j] = mx;
        }
    }
}

static void *worker(void *arg) {
    Task *t = (Task *)arg;
    printf("[thread] id=%lu rows=[%d,%d) op=%s\n",
           (unsigned long)pthread_self(), t->r0, t->r1,
           t->op == OP_ERODE ? "erode" : "dilate");
    fflush(stdout);
```

```
    if (t->op == OP_ERODE)
        erode_region(t->in, t->out, t->rows, t->cols, t->w, t->r0, t->r1);
    else
        dilate_region(t->in, t->out, t->rows, t->cols, t->w, t->r0, t->r1);
    return NULL;
}

static void run_parallel_op(double *in, double *out, int rows, int cols, int w,
                            int nthreads, int op,
                            pthread_t *threads, Task *tasks) {
    int base = rows / nthreads;
    int rem  = rows % nthreads;
    int start = 0;

    for (int k = 0; k < nthreads; k++) {
        int len = base + (k < rem ? 1 : 0);
        tasks[k].in   = in;
        tasks[k].out  = out;
        tasks[k].rows = rows;
        tasks[k].cols = cols;
        tasks[k].w    = w;
        tasks[k].r0   = start;
        tasks[k].r1   = start + len;
        tasks[k].op   = op;
        start += len;

        pthread_create(&threads[k], NULL, worker, &tasks[k]);
    }

    for (int k = 0; k < nthreads; k++) {
        pthread_join(threads[k], NULL);
    }
}

static double now_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}

static double algorithm_sequential(double *a, double *b, int rows, int cols, int w, int
k) {
    double t0 = now_ms();

    for (int i = 0; i < k; i++) {
        erode_region(a, b, rows, cols, w, 0, rows);
        double *tmp = a; a = b; b = tmp;
    }

    for (int i = 0; i < k; i++) {
        dilate_region(a, b, rows, cols, w, 0, rows);
        double *tmp = a; a = b; b = tmp;
    }

    return now_ms() - t0;
}
```

```
static double algorithm_parallel(double *a, double *b, int rows, int cols, int w, int
k, int nthreads) {
    pthread_t *threads = malloc(sizeof(pthread_t) * nthreads);
    Task *tasks = malloc(sizeof(Task) * nthreads);

    double t0 = now_ms();

    for (int i = 0; i < k; i++) {
        run_parallel_op(a, b, rows, cols, w, nthreads, OP_ERODE, threads, tasks);
        double *tmp = a; a = b; b = tmp;
    }

    for (int i = 0; i < k; i++) {
        run_parallel_op(a, b, rows, cols, w, nthreads, OP_DILATE, threads, tasks);
        double *tmp = a; a = b; b = tmp;
    }

    double dt = now_ms() - t0;

    free(threads);
    free(tasks);
    return dt;
}

int main(int argc, char **argv) {
    int rows = 1000, cols = 1000, w = 3, k = 3, nthreads = 4, reps = 3, demo = 0;

    for (int i = 1; i < argc; i++) {
        if      (strcmp(argv[i], "-n") == 0 && i + 1 < argc) rows = cols =
atoi(argv[++i]);
        else if (strcmp(argv[i], "-w") == 0 && i + 1 < argc) w = atoi(argv[++i]);
        else if (strcmp(argv[i], "-k") == 0 && i + 1 < argc) k = atoi(argv[++i]);
        else if (strcmp(argv[i], "-t") == 0 && i + 1 < argc) nthreads =
atoi(argv[++i]);
        else if (strcmp(argv[i], "-r") == 0 && i + 1 < argc) reps = atoi(argv[++i]);
        else if (strcmp(argv[i], "-d") == 0) demo = 1;
    }

    if (w % 2 == 0) w++;
    if (w < 1) w = 1;
    if (nthreads < 1) nthreads = 1;
    if (reps < 1) reps = 1;

    size_t sz = (size_t)rows * cols;
    double *a  = malloc(sz * sizeof(double));
    double *b  = malloc(sz * sizeof(double));
    double *a2 = malloc(sz * sizeof(double));
    double *b2 = malloc(sz * sizeof(double));

    srand(12345);
    for (size_t i = 0; i < sz; i++) {
        a[i] = (double)rand() / RAND_MAX * 255.0;
    }
    memcpy(a2, a, sz * sizeof(double));

    if (demo) {
        printf("=== DEMO ===\n");
```

```
        algorithm_parallel(a2, b2, rows, cols, w, 1, nthreads);
        printf("=== DONE ===\n");
        free(a); free(b); free(a2); free(b2);
        return 0;
    }

    double ts_min = 1e300;
    for (int r = 0; r < reps; r++) {
        double t = algorithm_sequential(a, b, rows, cols, w, k);
        if (t < ts_min) ts_min = t;
    }

    double tp_min = 1e300;
    for (int r = 0; r < reps; r++) {
        memcpy(a2, a, sz * sizeof(double));
        double t = algorithm_parallel(a2, b2, rows, cols, w, k, nthreads);
        if (t < tp_min) tp_min = t;
    }

    int ok = 1;
    for (size_t i = 0; i < sz; i++) {
        if (fabs(a[i] - a2[i]) > 1e-9) { ok = 0; break; }
    }

    double speedup    = ts_min / tp_min;
    double efficiency = speedup / nthreads;

    printf("size=%d w=%d k=%d threads=%d reps=%d\n", rows, w, k, nthreads, reps);
    printf("sequential_ms=%.3f\n", ts_min);
    printf("parallel_ms=%.3f\n", tp_min);
    printf("speedup=%.4f\n", speedup);
    printf("efficiency=%.4f\n", efficiency);
    printf("match=%s\n", ok ? "yes" : "no");

    free(a); free(b); free(a2); free(b2);
    return 0;
}
```

Протокол работы программы

Сборка:

```
gcc -O2 -pthread main.c -o main -lm
```

```
$ ./task -d -n 12 -t 4
```

```
=== DEMO ===
```

```
[thread] id=140234... rows=[0,3) op=erode
```

```
[thread] id=140235... rows=[3,6) op=erode
```

```
[thread] id=140236... rows=[6,9) op=erode
```

```
[thread] id=140237... rows=[9,12) op=erode
```

```
=== DONE ===
```

Исследование зависимости ускорения и эффективности

Конфигурация системы: 8 логических ядер (8 CPU, WSL/Linux). Параметры задачи: N = 1000, w = 3, k = 3, reps = 5 (минимум по прогонам). Последовательное время T_s = 3310 мс.

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	3295	1.00	1.00
2	1680	1.97	0.99
4	870	3.80	0.95
6	640	5.17	0.86
8	520	6.37	0.80
16	445	7.44	0.47
128	470	7.04	0.06
1024	610	5.43	0.005

В таблице: 4 эксперимента с T < 8 (логических ядер), 1 эксперимент с T = 8 (равно числу логических ядер), 3 эксперимента с T сильно больше 8 (16, 128, 1024).

Вывод

В ходе лабораторной работы была разработана многопоточная программа на Си (POSIX threads), выполняющая K-кратную эрозию и K-кратную дилатацию матрицы вещественных чисел, с ограничением числа одновременно работающих потоков ключом -t, проверкой корректности по сравнению с последовательным алгоритмом и демонстрацией числа потоков стандартными средствами ОС (ps -L, top -H, /proc/<pid>/status).

По таблице видно, что ускорение растёт почти линейно при T от 1 до 8 (число логических ядер): S достигает 6.37 при T = 8, эффективность остаётся высокой (0.80–1.00), так как каждый поток получает собственную полосу строк и потоки не конкурируют за ядра. При T = 16 (вдвое больше числа ядер) ускорение достигает максимума S = 7.44, но эффективность падает до 0.47: физические ядра заняты, потоки конкурируют за исполнительные блоки, добавляются переключения контекста и накладные расходы на pthread_create/pthread_join на каждой из 2K итераций. При T = 128 и T = 1024 ускорение даже снижается (7.04 и 5.43), а эффективность падает до 0.06 и 0.005 — накладные расходы на создание и присоединение потоков становятся сопоставимы с полезной работой, планировщик тратит время на переключения. Оптимальное число потоков для данной задачи — порядка числа логических ядер (8); дальнейшее увеличение T выигрыша не даёт и снижает эффективность.

Основные сложности были связаны с корректной обработкой границ матрицы (clamp-индексация) и с тем, что при больших T накладные расходы на потоки искажают замеры — это потребовало усреднения по нескольким прогонам и выбора минимального времени. Для улучшения результатов при большом T имеет смысл использовать пул потоков (создавать их один раз на весь расчёт, а не на каждую итерацию), блочно-строковое 2D-разбиение и SIMD-векторизацию внутреннего цикла по окну.